Resampling

1. Resampling 1D

```python
# -*- coding: utf-8 -*-
"""
Created on Mon Jan 19 17:51:11 2026

@author: reina
"""

import cv2
import numpy as np

class Upsample1d:
    def __init__(self, upsampling_factor):
        self.upsampling_factor = upsampling_factor

    def forward(self, x):
        N, C, input_width = x.shape
        output_width = self.upsampling_factor * (input_width - 1) + 1
        z = np.zeros(shape=(N, C, output_width), dtype=np.float64)
        for i in range(0, output_width, self.upsampling_factor):
            z[:, :, i] = x[:, :, i // self.upsampling_factor]
        return z

    def backward(self, dLdz):
        N, C, output_width = dLdz.shape
        input_width = (output_width - 1) // self.upsampling_factor + 1
        dLdx = np.zeros(shape=(N, C, input_width), dtype=np.float64)
        for i in range(0, output_width, self.upsampling_factor):
            dLdx[:, :, i // self.upsampling_factor] = dLdz[:, :, i]
        return dLdx

class Downsample1d:
    def __init__(self, downsampling_factor):
        self.downsampling_factor = downsampling_factor

    def forward(self, x):
        N, C, input_width = x.shape
        self.original_input_width = input_width
        output_width = (input_width - 1) // self.downsampling_factor + 1
        z = np.zeros(shape=(N, C, output_width), dtype=np.float64)
        for i in range(output_width):
            z[:, :, i] = x[:, :, i * self.downsampling_factor]
        return z

    def backward(self, dLdz):
        N, C, output_width = dLdz.shape
        dLdx = np.zeros(shape=(N, C, self.original_input_width), dtype=np.float64)
        for i in range(output_width):
            dLdx[:, :, i * self.downsampling_factor] = dLdz[:, :, i]
        return dLdx

def test_upsample1D(N, C, W, S):
    upsample = Upsample1d(S)
    # Insert S-1 columns between each original columns.
```

```
    # Therefore, the output will be of size W + (S-1)*(W-1).= S*(W-1) + 1
    # W = 5, S = 3, then output_size = 5 + 2 * 4 = 13.

    input_tensor = np.random.uniform(0, 5, (N, C, W))
    print("input_tensor.shape", input_tensor.shape)

    output_tensor = upsample.forward(input_tensor)
    print("output_tensor.shape", output_tensor.shape)

    dLdoutput_tensor = np.ones_like(output_tensor)
    dLdinput_tensor = upsample.backward(dLdoutput_tensor)

    print("input_tensor", input_tensor)
    print("output_tensor", output_tensor)
    print("dLdoutput_tensor", dLdoutput_tensor)
    print("dLdinput_tensor", dLdinput_tensor)

    print("input_tensor.shape", input_tensor.shape)
    print("output_tensor.shape", output_tensor.shape)
    print("dLdoutput_tensor.shape", dLdoutput_tensor.shape)
    print("dLdinput_tensor.shape", dLdinput_tensor.shape)

def test_downsample1D(N, C, W, S):
    downsample = Downsample1d(S)
    # Only keep columns with index S*i, where i=0,...,W//S.

    input_tensor = np.random.uniform(0, 5, (N, C, W))
    print("input_tensor.shape", input_tensor.shape)

    output_tensor = downsample.forward(input_tensor)
    print("output_tensor.shape", output_tensor.shape)

    dLdoutput_tensor = np.ones_like(output_tensor)
    dLdinput_tensor = downsample.backward(dLdoutput_tensor)

    print("input_tensor", input_tensor)
    print("output_tensor", output_tensor)
    print("dLdoutput_tensor", dLdoutput_tensor)
    print("dLdinput_tensor", dLdinput_tensor)

    print("input_tensor.shape", input_tensor.shape)
    print("output_tensor.shape", output_tensor.shape)
    print("dLdoutput_tensor.shape", dLdoutput_tensor.shape)
    print("dLdinput_tensor.shape", dLdinput_tensor.shape)

if __name__ == '__main__':
    N = 1
    C = 3
    W = 5
    S = 3
    test_upsample1D(N, C, W, S)
    N = 1
    C = 3
    W = 13
    S = 3
    test_downsample1D(N, C, W, S)
```

2. Resampling2D

```python
# -*- coding: utf-8 -*-
"""
Created on Fri Feb 13 15:01:25 2026

@author: reina
"""

import numpy as np

class Upsample2d:
    def __init__(self, upsampling_factor):
        self.upsampling_factor = upsampling_factor

    def forward(self, x):
        N, C, input_height, input_width = x.shape
        output_height = (input_height - 1) * self.upsampling_factor + 1
        output_width = (input_width - 1) * self.upsampling_factor + 1
        z = np.zeros(shape=(N, C, output_height, output_width), dtype=np.float64)
        for i in range(0, output_height, self.upsampling_factor):
            for j in range(0, output_width, self.upsampling_factor):
                z[:, :, i, j] = x[
                    :, :, i // self.upsampling_factor, j // self.upsampling_factor
                ]
        return z

    def backward(self, dLdz):
        N, C, output_height, output_width = dLdz.shape
        input_height = (output_height - 1) // self.upsampling_factor + 1
        input_width = (output_width - 1) // self.upsampling_factor + 1
        dLdx = np.zeros(shape=(N, C, input_height, input_width), dtype=np.float64)
        for i in range(0, output_height, self.upsampling_factor):
            for j in range(0, output_width, self.upsampling_factor):
                dLdx[:, :, i // self.upsampling_factor, j // self.upsampling_factor] = (
                    dLdz[:, :, i, j]
                )
        return dLdx


class Downsample2d:
    def __init__(self, downsampling_factor):
        self.downsampling_factor = downsampling_factor

    def forward(self, x):
        N, C, input_height, input_width = x.shape
        self.original_input_height = input_height
        self.original_input_width = input_width
        output_height = (input_height - 1) // self.downsampling_factor + 1
        output_width = (input_width - 1) // self.downsampling_factor + 1
        z = np.zeros(shape=(N, C, output_height, output_width), dtype=np.float64)
        for i in range(output_height):
            for j in range(output_width):
                z[:, :, i, j] = x[
                    :, :, i * self.downsampling_factor, j * self.downsampling_factor
                ]
        return z
```

```
    def backward(self, dLdz):
        N, C, output_height, output_width = dLdz.shape
        dLdx = np.zeros(
            shape=(N, C, self.original_input_height, self.original_input_width),
            dtype=np.float64,
        )
        #print("sampling dLdx.shape ", dLdx.shape)
        #print("sampling dLdz.shape ", dLdz.shape)
        for i in range(output_height):
            for j in range(output_width):
                dLdx[
                    :, :, i * self.downsampling_factor, j * self.downsampling_factor
                ] = dLdz[:, :, i, j]
        return dLdx

if __name__ == '__main__':
    # Upsample by a factor of S=2,
    # add S-1 rows of zeros and
    # S-1 columns of zeros
    up2d = Upsample2d(2)
    x = np.array([[[[1, 2],[3, 4]]]])
    z = up2d.forward(x)
    print("z.shape", z.shape)
    dLdz = np.random.random(z.shape)
    dLdx = up2d.backward(dLdz)
    print("dLdx.shape", dLdx.shape)

    # Upsample by a factor of S=5,
    # add S-1 rows of zeros and
    # S-1 columns of zeros
    up2d = Upsample2d(5)
    x = np.array([[[[1, 2],[3, 4]]]])
    z = up2d.forward(x)
    print("z.shape", z.shape)
    dLdz = np.random.random(z.shape)
    dLdx = up2d.backward(dLdz)
    print("dLdx.shape", dLdx.shape)

    # Then downsample
    ds2d = Downsample2d(5)
    x = np.random.random(z.shape)
    z = ds2d.forward(x)
    print("z.shape", z.shape)
    dLdz = np.random.random(z.shape)
    dLdx = ds2d.backward(dLdz)
    print("dLdx.shape", dLdx.shape)
```